

1. mpi_demo1.c

```
#include <mpi.h>
#include <stdio.h>
int main()
{
    MPI_Init(NULL, NULL);

    int np;
    MPI_Comm_size(MPI_COMM_WORLD, &np);

    int pid;
    MPI_Comm_rank(MPI_COMM_WORLD, &pid);

    printf("\n Hi lam processor%d out of %d",pid,np);

    MPI_Finalize();
    return 0;
}
```

Output

[vk@Prithivi ~]\$ lamboot

LAM 7.0.6/MPI 2 C++/ROMIO - Indiana University

[vk@Prithivi ~]\$ vi mpi_demo1.c

[vk@Prithivi ~]\$ mpicc mpi_demo1.c -o md1

[vk@Prithivi ~]\$ mpirun -np 5 md1

Hi lam processor0 out of 5
Hi lam processor1 out of 5
Hi lam processor2 out of 5
Hi lam processor3 out of 5
Hi lam processor4 out of 5

2. Send, Receive method – single data

```
#include<mpi.h>
#include<stdio.h>
int main(int argc,char *argv[])
{
    int a,b,c,r;
    MPI_Init(&argc,&argv);
    MPI_Status s;
    MPI_Comm_rank(MPI_COMM_WORLD,&r);

    if(r==0)
    {
        printf("\n Enter the value of a&b:");
        scanf("%d %d",&a, &b);
        MPI_Send(&a, 1, MPI_INT, 1, 111, MPI_COMM_WORLD);
        MPI_Send(&b, 1, MPI_INT, 1, 111, MPI_COMM_WORLD);
    }
}
```

```

        if(r==1)
        {
            MPI_Recv(&a, 1, MPI_INT, 0, 111, MPI_COMM_WORLD, &s);
            MPI_Recv(&b, 1, MPI_INT, 0, 111, MPI_COMM_WORLD, &s);
            c = a + b;
            printf("Sum=%d",c);
        }

MPI_Finalize();
return 0;
}

```

Output

```

mpicc sendrecv_demo1.c -o srd1
mpirun -np 2 srd1

```

Enter the value of a&b:10 20
Sum=30

3. Send, Receive method – multiple data

```

#include<mpi.h>
#include<stdio.h>
int main(int argc,char *argv[])
{
    int a[2],c,r;
    MPI_Init(&argc,&argv);
    MPI_Status s;
    MPI_Comm_rank(MPI_COMM_WORLD,&r);

    if(r==0)
    {
        printf("\n Enter two values :");
        scanf("%d %d",&a[0], &a[1]);
        MPI_Send(&a, 2, MPI_INT, 1, 111, MPI_COMM_WORLD);
    }

    if(r==1)
    {
        MPI_Recv(&a, 2, MPI_INT, 0, 111, MPI_COMM_WORLD, &s);
        c=a[0] + a[1];
        printf("Sum=%d",c);
    }

    MPI_Finalize();
    return 0;
}

```

Output

```

mpicc sendrecv_demo2.c -o srd2
mpirun -np 2 srd2

```

Enter two values : 5 6
Sum=11

Explore use of 'status'

```
#include<mpi.h>
#include<stdio.h>
int main(int argc,char *argv[])
{
    int a[2],c,r;
    MPI_Init(&argc,&argv);
    MPI_Status s;
    MPI_Comm_rank(MPI_COMM_WORLD,&r);

    if(r==0)
    {
        printf("\n Enter two values :");
        scanf("%d%d",&a[0],&a[1]);
        MPI_Send(&a, 2, MPI_INT, 1, 111, MPI_COMM_WORLD);
    }

    if(r==1)
    {
        MPI_Recv(&a,2,MPI_INT,MPI_ANY_SOURCE,MPI_ANY_TAG,MPI_COMM_WORLD,&s);
        printf("\n source = %d, tag = %d",s.MPI_SOURCE, s.MPI_TAG);

        int cnt;
        MPI_Get_count(&s,MPI_INT,&cnt);

        c=a[0] + a[1];
        printf("Sum=%d",c);
    }

    MPI_Finalize();
    return 0;
}
```

Output

```
mpicc sendrecv_demo2.c -o srd2
mpirun -np 2 srd2
```

Enter two values :5 6

source = 0, tag = 111
data count = 2

Sum=11

4. Broadcast method

```
#include<mpi.h>
#include<stdio.h>
int main(int argc,char **argv)
{
    MPI_Init(&argc,&argv);
```

```
int r,s;  
int src = 0, buffer;  
MPI_Comm_rank(MPI_COMM_WORLD,&r);  
if(r==0)  
{  
    buffer = 111;  
}  
MPI_Bcast(&buffer, 1, MPI_INT, src, MPI_COMM_WORLD);  
printf("\n Received %d by process %d",buffer,r);  
  
MPI_Finalize();  
return 0;  
}
```

Output

```
mpicc bcastdemo.c -o bd  
mpirun -np 4 bd
```

```
Received 111 by process 0  
Received 111 by process 1  
Received 111 by process 2  
Received 111 by process 3
```